

AMA - GPU (TD 3)

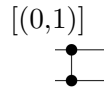
Jonàs Martínez

November 13, 2024

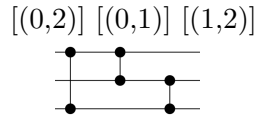
Exercise 1: Optimal networks

For small numbers of elements N , optimum sorting networks can be designed with little depth (i.e., calls to `EnqueueNDRangeKernel`). We list the first eight following the notation in https://bertdobbelaere.github.io/sorting_networks.html.

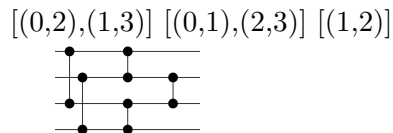
Size 2 (1 layer)



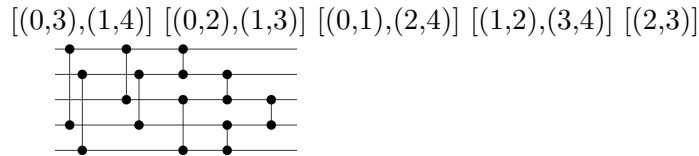
Size 3 (3 layers)



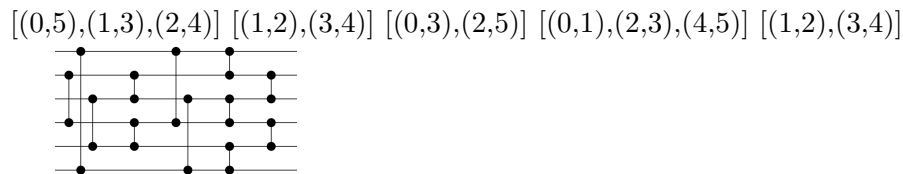
Size 4 (3 layers)



Size 5 (5 layers)

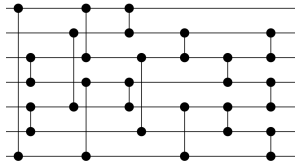


Size 6 (5 layers)



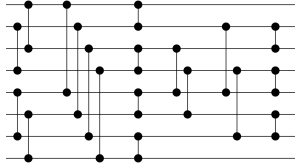
Size 7 (6 layers)

[(0,6),(2,3),(4,5)] [(0,2),(1,4),(3,6)] [(0,1),(2,5),(3,4)] [(1,2),(4,6)] [(2,3),(4,5)] [(1,2),(3,4),(5,6)]



Size 8 (6 layers)

[(0,2),(1,3),(4,6),(5,7)] [(0,4),(1,5),(2,6),(3,7)] [(0,1),(2,3),(4,5),(6,7)] [(2,4),(3,5)] [(1,4),(3,6)] [(1,2),(3,4),(5,6)]



Given an input array T with at most eight elements, implement an OpenCL algorithm that selects the corresponding optimal network for size N . As usual, evaluate the outcomes of a sequential CPU algorithm against your OpenCL implementation.

Exercise 2: Parallel Merge

We are given two ordered arrays (increasing value) of size N containing integers. We want to merge the two arrays into another ordered array of size $2N$. The arrays do not contain any repeated element for simplicity.

For example, given ordered arrays $A = [1, 4, 7, 13, 20]$ and $B = [2, 15, 21, 40, 63]$, we will obtain a merged ordered array $C = A \cup B = [1, 2, 4, 7, 13, 15, 21, 21, 40, 63]$.

Implement an OpenCL parallel algorithm to compute the merged array C .

The parallel merge must be performed with a single call to the kernel from the host.

Note Given a sorted array A and some number x that belongs to A or B , the function $low(x, A)$ is equal to the number of elements of A that are less than x . We can use this function to determine where the elements of A and B should be placed in the array C , considering that:

$$low(x, C) = low(x, A) + low(x, B)$$

In the example above, $low(15, A) = 4$ and $low(15, B) = 1$, and it follows that $low(15, C) = 5$. Thus, the value 15 must be placed in the $C[5]$ position of the array C . Note that the function $low(x, C)$ can be implemented in time $\log(N)$.